YOOBEE COLLEGE OF CREATIVE INNOVATION

Master of Software Engineering

Professional Software Engineering - MSE800

Assessment 1: Object-Oriented Programming

P2P Car Rental System

Lecturer: Dr. Mohammad Norouzifard

Student Name: Batbold Chuluunbaatar

Student ID: 270894331

Date: 24 May 2026

Abstract

This system is Python-based command line software for renting cars (P2P Car Rental System). Renters are connected to car owners through this system. There are three types of users: Owner, Renter, and Admin. The platform charges a service fee for every booking, and each one has its own menu.

The system operates through the basic principles of object-oriented programming which include encapsulation together with inheritance and abstraction and polymorphism. The system uses three design patterns which include Singleton and factory and strategy patterns. The system data storage through SQLite database implementation will bring about cost savings for operational activities. A maintenance plan and UML diagrams for the system are also provided with the project.

Table of Contents

1. Introduction

This document provides a description of the P2P Car Rental System, which is based on python-based command line application. It enables car owners to list their vehicles and then search and book them.An admin is responsible for verifying users and authorizing cars.

The platform operates in a P2P model without any cars being on hand. It links individuals with private owners. The owner of unreliable cars gains revenue from their vehicles. Renters are offered a greater range of choices, often at lower rates than those in traditional rental services.

Problem statement

The system (P2P) service must address multiple issues together.

- The platform will allow the service after verifying the user. This should influence trust in the user.
- Listings of cars must be reviewed before they appear in search results.
- The functioning of booking requires the interval of dates, pricing, and approvals.
- Risk can be reduced through insurance services
- All users should be able to rate and view ratings from others based on the services they have received.

Objectives

Objective	**How it is achieved**
OOP principles	Abstract User class with Owner, Renter, and Admin subclasses. Protected attributes with @property access. Polymorphic dashboard method.
Design patterns	Singleton for the database. Factory for user creation. Strategy for insurance
Layered architecture	Four layers: Presentation, Service, Model, and Data Access. Each layer communicates only with the one below it.
Environment support	config.py can read APP_ENV and switches between dev, test, and prod settings, depends on purpose.
Documentation	UML diagrams: ERD, Use Case, Class, Sequence, and Activity diagrams.

2. System requirements

Software requirements

Software	Version	Purpose
Python	3.8 or higher	Programming language.
SQLite	Built into Python	Database and no separate installation needed.
tabulate	0.9.0	Prints formatted tables in the CLI.
colorama	0.4.6	Adds color to CLI output.

No separate database server is needed. SQLite saves all data in one file: database/car_rental.db. The database file is created automatically and seed data (admin account) inserted on the first run.

Hardware requirements

Component	Minimum
CPU	1 GHz or faster
RAM	1GB
Disk space	1GB free
Display	80-column terminal or wider

3. Installation and configuration

Step 1: Extract the project source files

Extract the zip file (source_code.zip) to any folder and open a terminal and navigate to that folder.

Step 2: Create and activate a virtual environment

```
python -m venv venv
```

On Windows:

```
venv\Scripts\activate
```

On macOS or Linux:

```
source venv/bin/activate
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Run the application

```
python main.py
```

On first run, the system creates the database, sets up all tables, creates the default admin account, and shows the main menu.

Configuration

config.py contains all settings. The APP_ENV variable selects the environment. If not set, dev is used as default.

Environment	Database	Purpose
dev (default)	database/car_rental.db	For development, data is saved between runs.
test	:memory:	For testing, Data is deleted when the app closes.
prod	database/car_rental_prod.db	For production use.

To switch environment:

```
APP_ENV=test python main.py
```

Key configuration values

Constant	Default	Description
PLATFORM_FEE_PERCENT	15%	Commission on each booking for platform.
LONG_TERM_DISCOUNTS	7d=10%, 30d=20%	Discount conditions for longer rentals.
DISCOUNT_CODES	SAVE10, WELCOME20, SUMMER15	Available discount codes.
DEFAULT ADMIN_EMAIL	admin@p2p.com	Auto created admin login.

4. User guide

The system has three roles including renter, owner, and admin. Each role has its own menu. Test all features in the order below.

Step	Action	Role
1	Login as admin (admin@p2p.com / admin123).	Admin
2	Register a new owner account, then log out.	-
3	Register a new renter account, then log out.	-

4	Login as admin and verify both users.	Admin
5	Login as owner and add an own car.	Owner
6	Login as admin and approve the car.	Admin
7	Login as renter, search for cars, and make a booking.	Renter
8	Login as owner and approve the booking.	Owner
9	Login as renter and submit a review.	Renter

Owner features

- Add car listing: Enter and save make, model, year, mileage, location, daily rate, and min and max rent periods. The listing approved by admin approval.
- Manage bookings: view pending requests and approve or reject each one.
- View earnings: see total income from completed bookings.

Renter features

- Search cars: filter by location and maximum daily price.
- Make booking: Selecting the rental period and calculating the total price, select an insurance options and enter a discount code.
- Cancel booking: allowed before the rental start date.
- Submit review: rate the owner from 1 to 5 stars after the rental ends.

Admin features

- Verify users: check that new registrations are real people (KYC).
- Approve cars: review and approve or reject new car listings.
- Handle claims: review insurance claims and decide to approve, partially pay, or reject.
- View reports: see total users, cars, bookings, and platform revenue.
- Suspend users: remove verification from users who break the rules.

Calculation of price

- Base price = daily rate x number of days
- Long-term discount: 7 or more days = 10% off. 30 or more days = 20% off.
- Discount code: optional extra percentage off.
- Platform fee = 15% of the owner amount (service fee of the system).
- Insurance fee = daily insurance rate x number of days (optional).

Insurance options	Daily fee	Coverage
None	$0.00	No coverage. Renter accepts all risk.
Basic	$5.00	$2,000 coverage for minor damage.
Standard	$10.00	$5,000 coverage.
Premium	$15.00	Full coverage.

Discount codes

Discount codes can be entered during the booking process. They apply a percentage off the base price after the long-term discount is applied.

Code	Discount
SAVE10	10% off the base price
WELCOME20	20% off the base price
SUMMER15	15% off the base price

5. Project structure

The project uses a 4-layer architecture. Each folder has one responsibility.

Folder / File	Layer	Responsibility
main.py	Entry point	Initializes the database and starts the main menu.
config.py	Configuration	All settings. Reads APP_ENV to switch environments.
database/db_manager.py	Data Access	Singleton database manager. Runs all SQL queries.
models/	Model	OOP entities: User (abstract), Owner, Renter, Admin, Car, Booking, Review.
factories/user_factory.py	Model	Creates the correct User subclass from a role string.
patterns/insurance_strategy.py	Model	Four insurance classes sharing one interface.
services/	Service	Business logic: AuthService, CarService, BookingService, AdminService, ReviewService.
ui/	Presentation	CLI menus: main_menu, owner_menu, renter_menu, admin_menu.

utils/	Helper	security.py (password hashing), validators.py, helpers.py (display functions).

6. Design patterns

The system uses three design patterns, each one was chosen because it solves a specific problem in the code.

Singleton: DatabaseManager

If different parts of the code each open their own database connection, memory is wasted and write conflicts can occur. The DatabaseManager (database/db_manager.py) solves this by overriding __new__. Every call to DatabaseManager() returns the same object. Only one connection is ever created.

```
class DatabaseManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialize()
        return cls._instance
```

All parts of the application share this one connection. The database tables are also created only once, when the connection is first opened.

Factory: UserFactory

When a user logs in, the system needs to create an Owner, Renter, or Admin object depending on the role stored in the database. UserFactory puts all creation logic in one place (factories/user_factory.py). The factory also has a from_db_row() method that creates a user object directly from a database row.

```
class UserFactory:
    @staticmethod
    def create(role, user_id, name, email, ...):
        if role == "owner":
            return Owner(user_id, name, email, ...)
        elif role == "renter":
            return Renter(user_id, name, email, ...)
        elif role == "admin":
            return Admin(user_id, name, email, ...)
```

Strategy: InsuranceStrategy

The system offers four insurance options (patterns/ insurance_strategy.py). Each has a different daily fee and coverage limit. The Strategy pattern lets each plan be a separate class. The BookingService (services/ booking_service.py) calls insurance.calculate_fee(days) without knowing which plan is active. To add a new plan, only a new subclass is needed.

```
class InsuranceStrategy(ABC):
    @property
    @abstractmethod
    def daily_fee(self) -> float: pass

    @abstractmethod
    def calculate_fee(self, days: int) -> float: pass
```

```
class BasicInsurance(InsuranceStrategy):
    daily_fee = 5.0   # $2,000 coverage
```

```
class StandardInsurance(InsuranceStrategy):
    daily_fee = 10.0  # $5,000 coverage
```

```
class PremiumInsurance(InsuranceStrategy):
    daily_fee = 15.0  # Full coverage
```

Pattern summary

Pattern	Problem solved	File
Singleton	One database connection shared across the whole app.	database/db_manager.py
Factory	Create the right User subclass from a role string.	factories/user_factory.py
Strategy	Swap insurance pricing at runtime without changing the booking logic.	patterns/insurance_strategy.py

7. System Design and Architecture

7.1 Architecture overview

The system has a 4-layer architecture that communicate with the layer directly below them.

Layer	Folder	Responsibility
1. Presentation	ui/	CLI menus for each user role.
2. Service	services/	Business logic: booking, auth, admin operations.
3. Model	models/	OOP entities: User, Car, Booking, Review.
4. Data Access	database/	SQLite connection and all SQL queries.

7.2 Entity Relationship Diagram

The database contains five tables: USERS, CARS, BOOKINGS, REVIEWS, and INSURANCE_CLAIMS. All three user types are stored in the USERS table, using a role column to differentiate between them. The owner_id is stored directly in the BOOKINGS table to allow for faster queries of which owner booked the car. The discount_code and discount_amount are also stored in the BOOKINGS table to allow for auditing of any discounts applied to a booking. Finally, CHECK constraints are used on all status columns to ensure that only valid values are entered into those columns at the database level.

Table	Primary Key	Foreign Keys	Key Constraints
USERS	id	none	email UNIQUE, role CHECK (owner/renter/admin)
CARS	id	owner_id -> USERS	is_approved BOOL, daily_rate REAL
BOOKINGS	id	car_id, renter_id, owner_id	status CHECK (5 values), discount_code, discount_amount
REVIEWS	id	booking_id, reviewer_id, reviewee_id	rating CHECK (1-5)
INSURANCE_CLAIMS	id	booking_id	status CHECK (pending/approved/rejected)

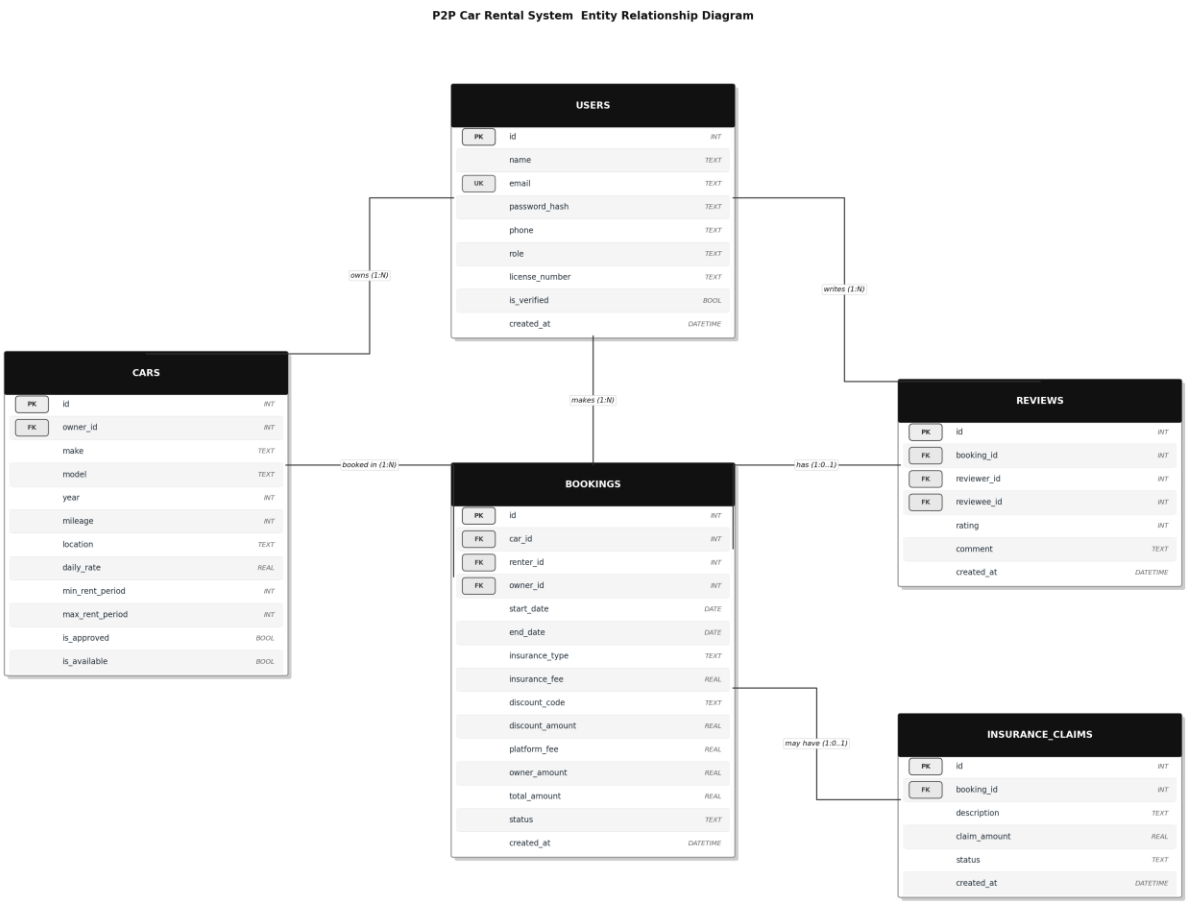

Figure 1: ERD with 5 tables, 8 foreign keys, and 1 unique key

7.3 Use Case Diagram

The system has three actors: Owner, Renter, and Admin. Two relationship types are used.

- "include": a mandatory step that always runs. "Make booking" always includes "Check availability" and "Calculate total fee".
- "extend": an optional step. "Make booking" can extend with "Select insurance" or "Apply discount code".

Actor	Use Cases
Owner	Register/login, add car listing, remove car listing, approve booking, reject booking, view earnings.
Renter	Register/login, Search for available cars by date periods, view car details, make booking, cancel booking, submit review.

Admin	Verify user KYC, approve car listing, reject car listing, view all bookings, handle insurance claim, view reports, suspend user.

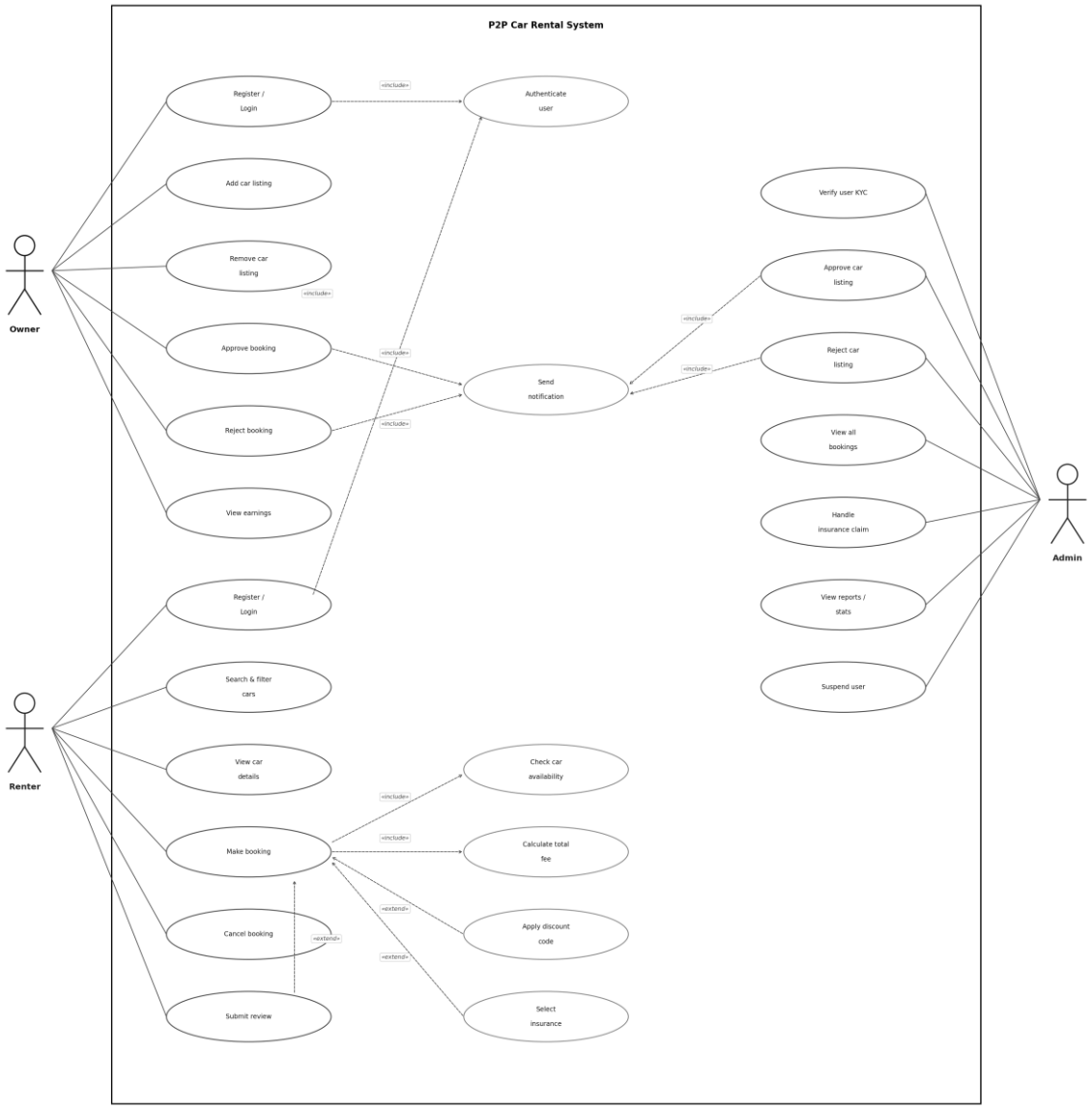

Figure 2: Use Case Diagram with 3 actors and 19 use cases

7.4 Class Diagram

The User class is abstract. It uses abc.ABC and defines view_dashboard() as an abstract method. Owner, Renter, and Admin each inherit from User and provide their own version of this method.

Principle	How it is applied
Encapsulation	All attributes use a single underscore prefix (_id, _name). They are accessed only through @property methods.
Abstraction	User and InsuranceStrategy are abstract base classes (ABC). They define the interface but not the implementation.
Inheritance	Owner, Renter, and Admin inherit all attributes and methods from User.
Polymorphism	view_dashboard() is defined once in User but works differently for each subclass. The main menu calls it without knowing the role.

Figure 3: Class Diagram showing the OOP hierarchy and design patterns

7.5 Sequence Diagram: Make booking

The Make booking use case is the most complex in the system. The sequence diagram shows 9 participants and the messages between them in order.

Step	Action	Notes
1	Renter logs in	Includes: Authenticate user
2	Renter searches for cars	SQL query with location and price filters
3	Renter views car details	Get one car record by ID
4	Renter enters booking dates	is_valid_date_range() is called first
5	System is checking availability	Include: Check car availability
6	System calculates the fee	Include: Calculate total fee
7	Renter selects insurance	Extend (optional)
8	Renter applies a discount code	Extend (optional)
9	System saves the booking	INSERT into BOOKINGS with status = pending
10	System notifies the owner	Include: Send notification

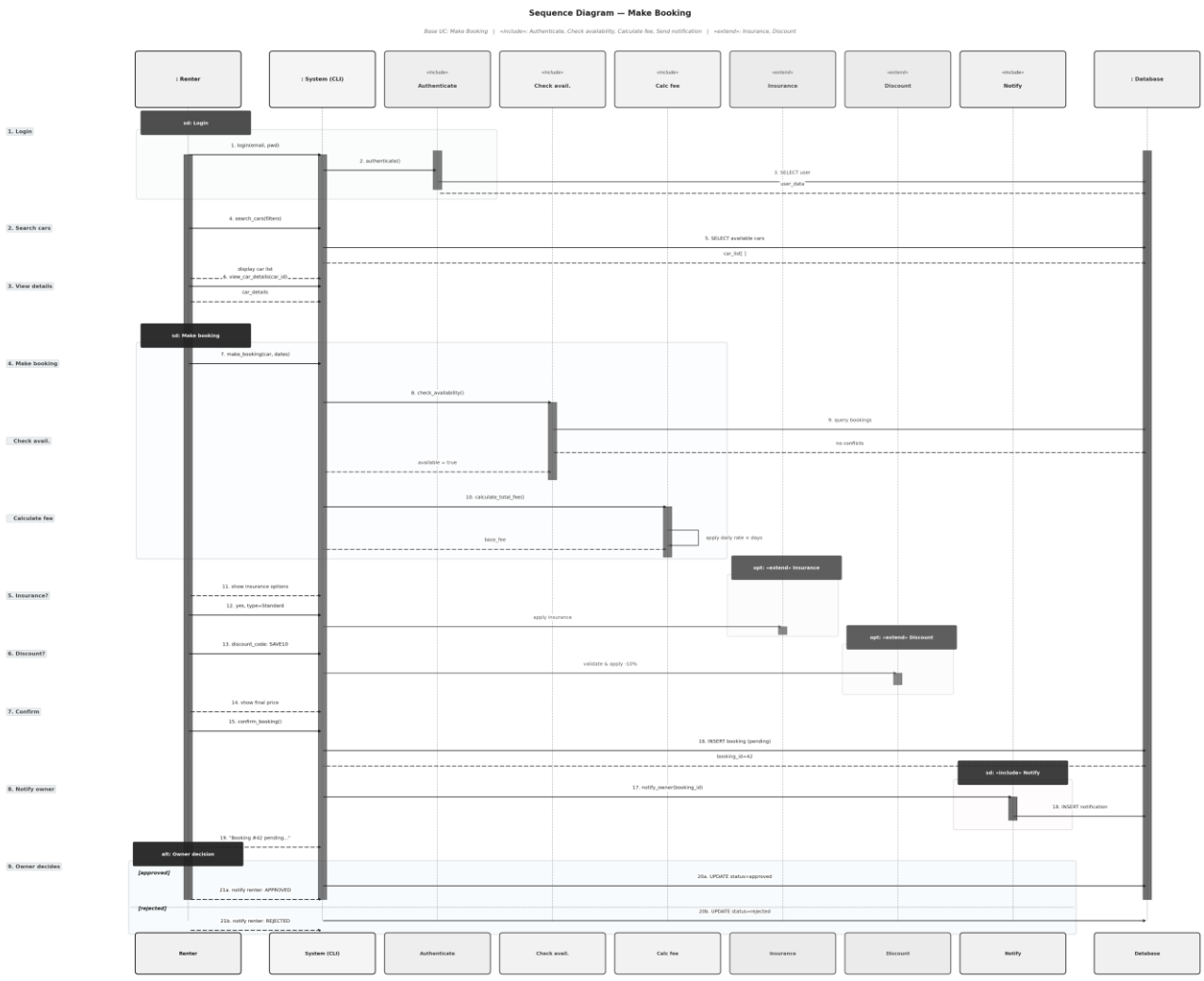

Figure 4: Sequence Diagram for the Make Booking use case

7.6 Activity Diagrams

Four activity diagrams show the step-by-step flow for the main use cases. Each uses decision nodes, fork and join bars for parallel steps, and start and end nodes.

Make booking

After a renter enters the dates for their booking, two steps run in parallel using a fork. The system checks if the car is available for those dates and calculates the total fee for the booking at the same time. The renter then has the option to add insurance or a discount code. If the renter confirms the booking, the details are saved.

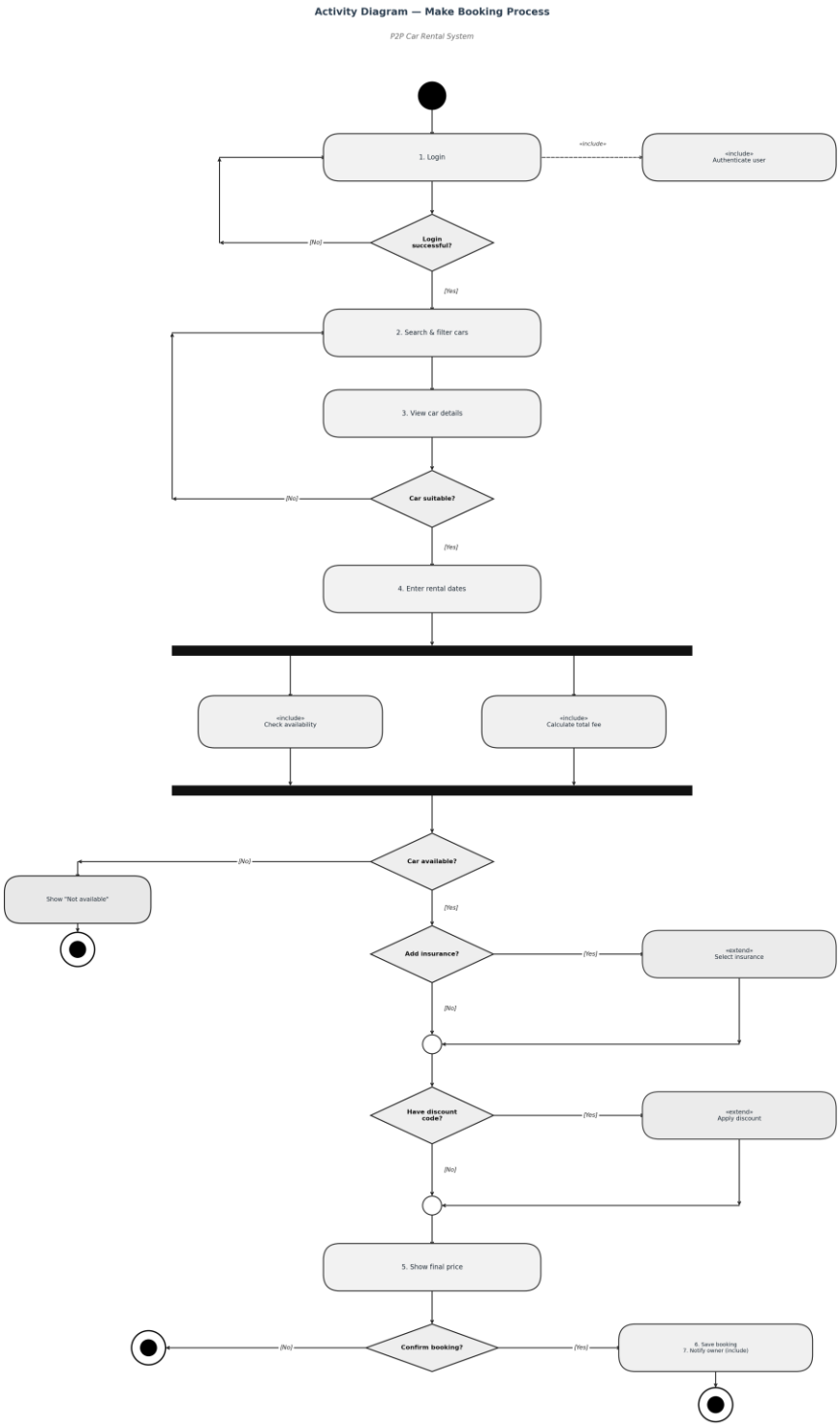

Figure 5: Activity Diagram for the Make Booking process

Add car listing

The diagram includes two separate swimlanes which separate Owner activities from Admin activities. The owner enters car details and submits the listing. The system stores the information under pending status while it sends an alert to the admin team. The admin team reviews the

matter before they choose to approve or reject it. The system shows the car in search results after it receives approval.

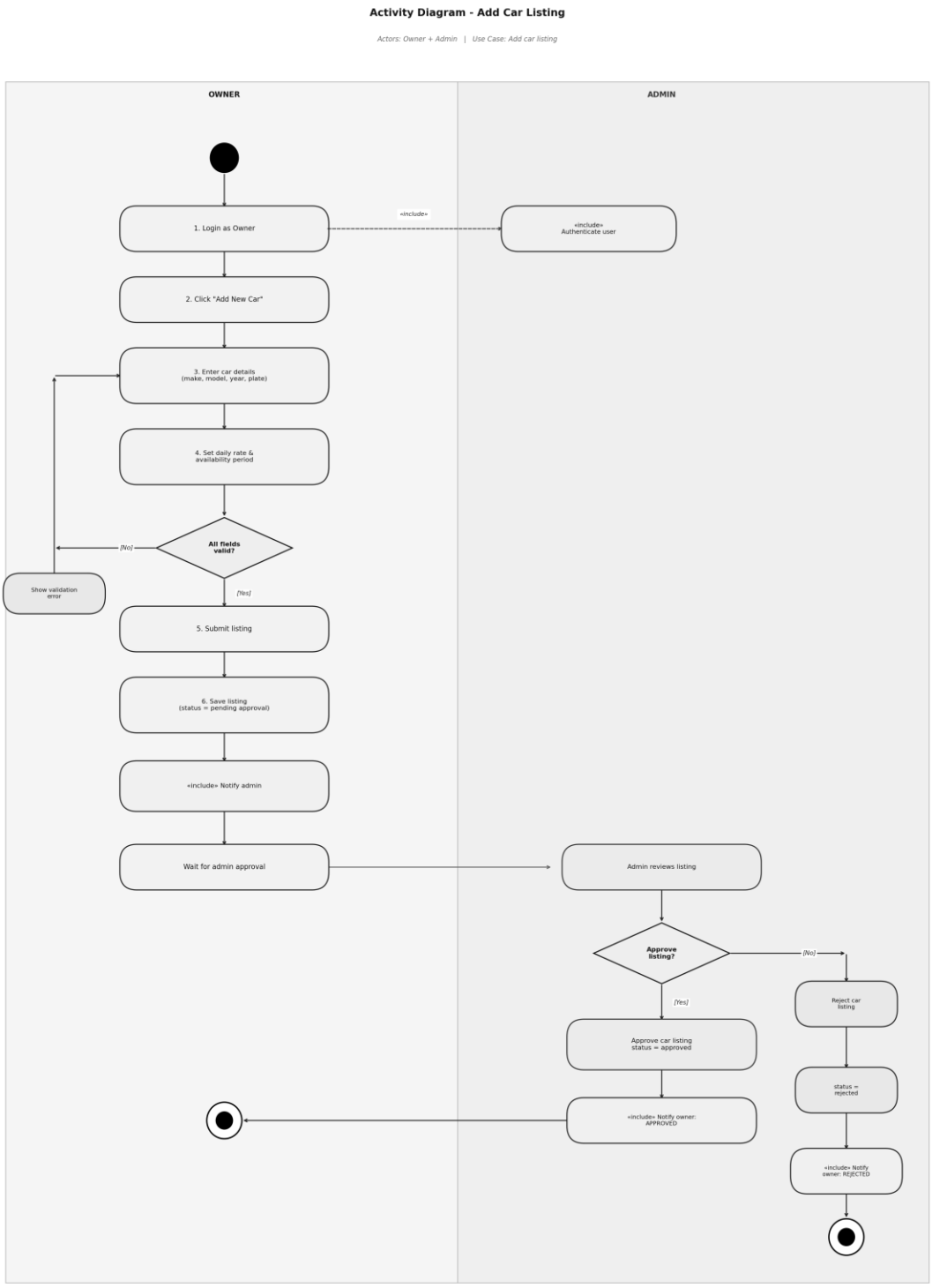

Figure 6: Activity Diagram for adding a car listing

Approve or reject booking

The owner logs into the application and views the pending booking requests. The owner checks the rating and booking history of the renter before making the decision. If the owner approves the booking, the status is updated to approved and the renter is notified. If the owner rejects the booking, the owner is asked for the reason for rejection and the status is updated to rejected.

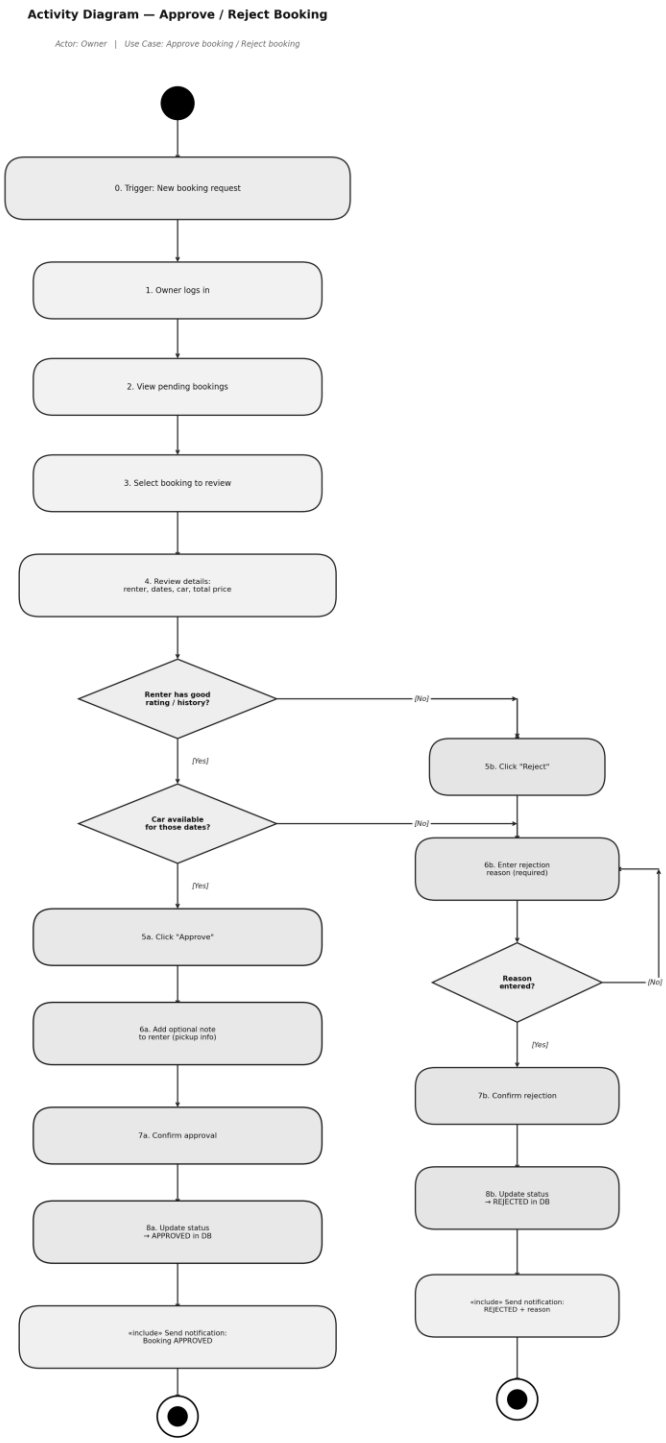

Figure 7: Activity Diagram for approving or rejecting a booking

Handle Insurance Claim

The diagram consists of three separate swimlanes which represent Owner and Admin and Renter functions. The owner files a claim after the rental. The evidence submission process requires both owners and renters to send their evidence at the same time through the fork system. The admin reviews all submitted evidence to make a decision between full claim approval or partial payment or complete claim deny.

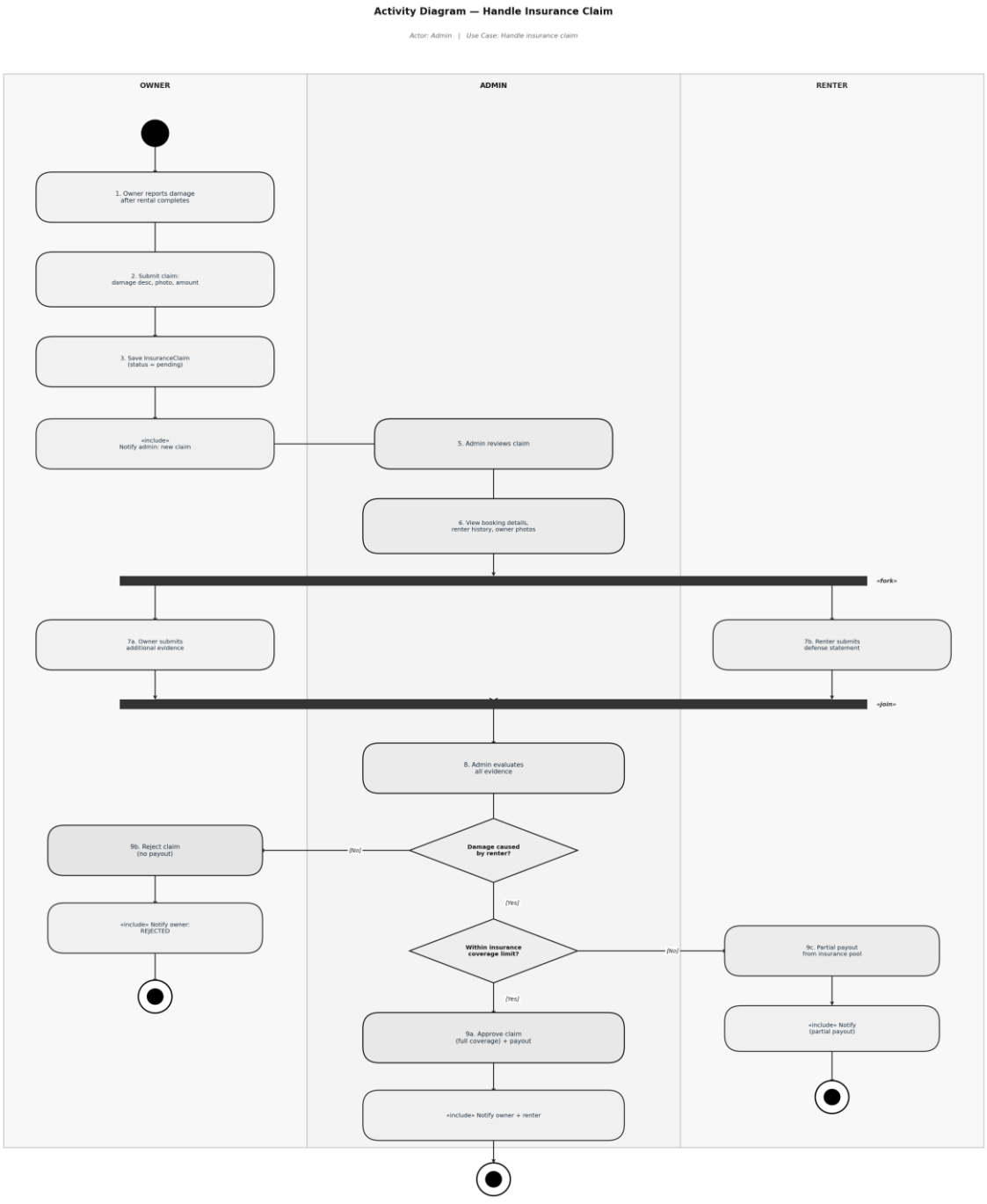

Figure 8: Activity Diagram for handling an insurance claim

8. Testing

The system was manually tested using integration tests, with each test checks one full feature from start to end and as in result all 15 test cases passed.

#	**Action/scenario**	**Result**
1	Default admin account created on first run.	PASS
2	Register with a duplicate email address.	PASS (rejected)
3	Admin verifies a pending user.	PASS
4	Owner adds a car and admin approves it.	PASS
5	Renter searches by location and maximum price.	PASS
6	Renter makes a 10-day booking with insurance and discount.	PASS
7	Pricing is calculated correctly (see below).	PASS
8	Date overlap detected. Conflicting booking rejected.	PASS
9	Owner approves a booking.	PASS
10	Renter submits a review with rating 1 to 5.	PASS
11	Renter submits a rating outside 1 to 5.	PASS (rejected)
12	Admin views platform statistics.	PASS
13	Login with wrong password.	PASS (rejected)
14	Renter cancels a booking before the start date.	PASS
15	Renter tries to cancel a completed booking.	PASS (rejected)

Pricing verification

Toyota Camry, $60/day, 10 days, Standard insurance, SAVE10 code:

Step	**Value**
Base price (10 x $60.00)	$600.00
Long-term discount (7+ days = 10% off)	-$60.00
SAVE10 discount code (10% off)	-$54.00
Owner amount	$486.00
Platform fee (15% of owner amount)	+$72.90

Standard insurance ($10/day x 10 days)	+$100.00
Total paid by renter	$658.90

The system output matched this calculation exactly.

9. Maintenance and Support plan

After the release of the system, it must be maintained to confirm it continues to function correctly and address user's needs. There are three main areas of maintenance discussed in this section including software maintenance, versioning, and backward compatibility.

Maintenance is one aspect of software development costs. Studies have shown that the cost of maintenance for software can be higher than the cost of development of the software (Haleem et al., 2025). Having a plan for software maintenance helps to manage these costs for the project from the beginning.

9.1 Software maintenance

The ISO/IEC/IEEE 14764 standard (ISO/IEC/IEEE, 2022) defines four types of software maintenance.

Type	Description	Example
Corrective	Fixing bugs found after release.	Fixing the date validation logic in the booking flow.
Adaptive	Updating for new environments.	Adding support for a new Python version.
Perfective	Improving or adding features.	Sorting car search results by rating.
Preventive	Reducing future problems.	Refactoring BookingService to lower complexity.

Issue Handling Process

1. The user reports the issue through email or the issue tracker.
2. The team assigns a severity level to the issue.
3. A ticket is created detailing the issue and how to reproduce it.
4. A developer is assigned to the ticket and creates a fix in a separate branch.
5. A second developer reviews the code.
6. All tests are run to ensure nothing is broken.
7. The fix is merged and a new release is published.
8. The changelog is updated.

Severity levels and response times

Severity	Example	Response time	Fix time
Critical	System is down or data is lost.	1 hour	4 hours
High	A main feature does not work.	2 hours	8 hours
Medium	Minor bug with a workaround.	12 hours	24 hours
Low	Visual issue or typo.	3 days	Next release

Code quality tools

- Git: version control and every change is tracked.

These tools are recommended for future development:

- Pytest: unit and integration tests (docs.pytest.org/en/stable/)
- Black: consistent code formatting (black.readthedocs.io)
- Flake8: checking common Python mistakes (flake8.pycqa.org)
- GitHub Actions: automated testing on every commit (github.com/features/actions)

9.2 Versioning

The project uses Semantic Versioning 2.0.0 (Preston-Werner, 2013). Each version number has three parts: MAJOR.MINOR.PATCH.

Part	When to increase	Example
MAJOR	A change breaks backward compatibility.	v1.0.0 becomes v2.0.0
MINOR	New feature added. Old features still work.	v1.2.0 becomes v1.3.0
PATCH	Bug fixed. No features added or removed.	v1.2.3 becomes v1.2.4

Release schedule

Type	Frequency	Content
Patch (1.0.x)	As needed	Bug fixes only.
Minor (1.x.0)	Every 4 to 6 weeks	New features. No breaking changes.
Major (x.0.0)	Every 12 to 18 months	Large changes. May break old behavior.

Git tags and changelog

Every release is tagged in Git. A CHANGELOG.md file records all changes following the Keep a Changelog standard (Oliver, 2017). Each entry has four sections: Added, Changed, Fixed, and Removed.

```
git tag -a v1.0.0 -m "Initial release"
git push origin v1.0.0
```

9.3 Backward compatibility

Backward compatibility means new versions can still work with data from older versions.

Type	Description	Example
Data compatibility	New code can read old database files.	A v1.0 database still works in v1.5 after migration.
API compatibility	Old function names and parameters still work.	register(name, email) works the same in v2.0.
User experience	Menu structure stays similar between versions.	Existing users do not need to relearn the system.

Database migrations

When the database schema changes, a migration script will update the old database without deleting any data. These scripts are numbered and stored within the database/migrations/ directory. Each script uses the command ALTER TABLE ... ADD COLUMN IF NOT EXISTS to allow the script to be run more than once without causing any issues to the database.

Deprecation lifecycle

Stage	What happens	Duration
1. Announce	Users are told the feature will be removed.	6 months before removal
2. Coexist	Old and new features both work. Old one shows a warning.	3 to 6 months
3. Remove	Old feature is removed from the code.	In the next major version

Future roadmap

Version	Target	Planned changes
v1.0.0	Current	Initial release with all core features.
v1.1.0	Q3 2026	Email notifications via SMTP.
v1.2.0	Q4 2026	Loyalty points and tier discounts.
v1.3.0	Q1 2027	Multi-language support.
v2.0.0	Q2 2027	PostgreSQL backend and REST API.
v2.1.0	Q3 2027	Mobile app (React Native).

v2.2.0	Q4 2027	Stripe payment integration.
v3.0.0	Q2 2028	Smart matching with machine learning.

10. Conclusion

This project brought together OOP principles, design patterns, and architectures can be implemented into a single Python application.

All of the main OOP principles are demonstrated within the project. Data encapsulation is used to protect the data within each of the model classes. Inheritance is used to create a clean user hierarchy for the application. Abstraction hides the implementation details of the application behind interfaces. Polymorphism allows for the application to handle the different user roles and insurance types without the use of additional conditional statements.

Three design patterns are implemented within the project to solve various problems. The Singleton design pattern is used to control access to the database. The Factory design pattern removes the need to repeat the logic for creating users for the application. The Strategy design pattern makes it easy to add new insurance pricing algorithms for the application.

The application uses a 4-layer software architecture to keep the code organized. Each layer of the architecture has a specific job and does not depend upon the layers above it in the architecture. The application supports three different environments through a single configuration file. Eight UML diagrams are included to document the application’s structure, behavior, and process flows. The maintenance plan for the application covers all four types of software maintenance, semantic versioning, backward compatibility, and includes a 24-month development roadmap for the application.

11. References

Haleem, M., Islam, M., Ahmed, M. N., Farooqui, N. A., Khan, A. N., Hussain, M. R., Ahmed, J. S., Ahmed, M. M., and Khan, I. M. (2025). A Critical Review for Software Maintenance Cost Estimation Models: A Data Driven Approach. IEEE Access. https://doi.org/10.1109/ACCESS.2025.3618759

ISO/IEC/IEEE (2022). ISO/IEC/IEEE 14764:2022 Software Engineering: Software Life Cycle Processes - Maintenance (Edition 3). International Organization for Standardization. https://www.iso.org/obp/ui/en/#iso:std:iso-iec-ieee:14764:ed-3:v1:en

Oliver, O. (2017). Keep a Changelog 1.0.0. https://keepachangelog.com/

Preston-Werner, T. (2013). Semantic Versioning 2.0.0. https://semver.org/

Appendix: Default credentials

On the first run, the system creates a default admin account automatically.

Field	Value
Email	admin@p2p.com
Password	admin123
Role	Admin